HEAT: NPU-NDP HEterogeneous Architecture for Transformer-Empowered Graph Neural Networks

Ruiyang Chen
Shanghai Jiao Tong University
Shanghai, China
chenruiyang@sjtu.edu.cn

Zhuoran Song*
Shanghai Jiao Tong University
Shanghai, China
songzhuoran@sjtu.edu.cn

Yicheng Zheng
Shanghai Jiao Tong University
Shanghai, China
bhunterb@sjtu.edu.cn

Zeyu Zhu
Institute of Automation, Chinese Academy of Sciences
Beijing, China
zhuzeyu2021@ia.ac.cn

Gang Li
Institute of Automation, Chinese Academy of Sciences
Beijing, China
gang.li@ia.ac.cn

Naifeng Jing
Shanghai Jiao Tong University
Shanghai, China
sjtuj@sjtu.edu.cn

Xiaoyao Liang
Shanghai Jiao Tong University
Shanghai, China
liang-xy@cs.sjtu.edu.cn

Haibing Guan
Shanghai Jiao Tong University
Shanghai, China
hbguan@sjtu.edu.cn

Abstract

Transformer-empowered Graph Neural Networks (TF-GNNs) are gaining significant attention in AI research because they leverage the front-end Transformer's ability to process textual data while also harnessing the back-end GNN's capacity to analyze graph structures. Typically, TF-GNNs follow the sequential execution mode, where the front-end Transformer first encodes vertex features, followed by subgraph sampling and subsequent processing by the back-end GNN. However, due to the massive computation workloads of Transformers and the irregular memory access patterns of GNNs, achieving efficient inference for TF-GNNs remains a challenge. Although architectures like FACT and MEGA have been proposed to separately accelerate the Transformer and GNN, they overlook the new opportunities arising from the coupling of the Transformer and GNN.

To enable efficient TF-GNNs, we propose HEAT, a heterogeneous architecture with a Neural Processing Unit (NPU) and a DIMM-based Near-Data Processing (NDP). Such a heterogeneous architecture can utilize both the high computational power of NPU and the high internal bandwidth of NDP. To fully unleash the potential of the NPU-NDP architecture, HEAT makes the following three contributions: First, HEAT leverages graph topology to identify the importance of vertices and encodes their features in the Transformer using varying precision accordingly. Second, HEAT gives more flexibility to the execution granularity and execution order of subgraphs in GNN, allowing their DRAM accesses to scatter across different banks and enhancing locality between subgraphs. Third, HEAT decouples the inherent dependency between the front-end Transformer and the back-end GNN by allowing the Transformer to selectively encode only the vertex features required by the current subgraph. This enables concurrent execution of the NPU and NDP, thereby fully exploiting the parallelism of the heterogeneous architecture. Comprehensive evaluations on a wide range of datasets demonstrate that HEAT outperforms the high-performance A100 GPU, the state-of-the-art (SOTA) Transformer accelerator FACT, and GNN accelerator MEGA in terms of execution time and energy efficiency.

Keywords

TF-GNN, Graph, Heterogeneous Architecture, NPU, NDP

ACM Reference Format:
Ruiyang Chen, Zhuoran Song, Yicheng Zheng, Zeyu Zhu, Gang Li, Naifeng Jing, Xiaoyao Liang, and Haibing Guan. 2025. HEAT: NPU-NDP HEterogeneous Architecture for Transformer-Empowered Graph Neural Networks. In *58th IEEE/ACM International Symposium on Microarchitecture (MICRO '25), October 18–22, 2025, Seoul, Republic of Korea.* ACM, New York, NY, USA, 14 pages. https://doi.org/10.1145/3725843.3756117

1 Introduction

Recently, Transformer-empowered Graph Neural Networks (TF-GNNs[8, 18, 33, 36, 64]) have emerged as a hot topic in AI research. By combining the strengths of Transformers in processing textual data and the analytical capabilities of Graph Neural Networks (GNNs) in handling graph structures, TF-GNNs have demonstrated exceptional performance across diverse domains, including social networks [10, 28], recommendation systems [23, 56], and bioinformatics [32]. TF-GNNs mainly consist of two phases: the front-end, where the Transformer encodes tokens from the input Text-attributed Graph (TAG) to generate vertex features, and the back-end, where subgraphs are sampled from the TAG, and the generated

*Zhuoran Song is the corresponding author.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

MICRO '25, Seoul, Republic of Korea

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 979-8-4007-1573-0/25/10
https://doi.org/10.1145/3725843.3756117

Table 1: The comparison of HEAT with other state-of-the-art accelerators.

Solutions	Optimize Transformer	Optimize GNN	Optimize Both	Memory-efficiency in GNN ↑
SpAtten [53]	✓	✗	✗	Low
FACT [40]	✓	✗	✗	Low
SGCN [57]	✗	✓	✗	Medium
MEGA [65]	✗	✓	✗	Medium
HEAT	✓	✓	✓	High

vertex features are input into the GNN for final classification results. Due to the reliance of the back-end GNN on the vertex features generated by the front-end Transformer, there is an inherent algorithmic dependency between them.

Although TF-GNNs have demonstrated a significant performance improvement over traditional GNNs in numerous applications, reaching a 30.79% improvement in accuracy within the knowledge graph field [36], it comes at the cost of over 1000× increased latency and energy consumption. This presents a huge challenge for deploying TF-GNNs in real-world scenarios. Through an in-depth analysis of TF-GNNs' performance on GPUs, we identify that the major bottlenecks lie in both the compute-bound Transformer and the memory-bound GNN, which together create challenges in both computation and memory.

To address the memory-bound challenges in traditional GNNs, several GNN accelerators have been proposed [5, 6, 11, 12, 22, 30, 34, 35, 44, 55, 57, 58, 60, 65], most of which optimize the data format of the vertex features to reduce memory access overhead. For instance, SGCN [57] introduces a novel vertex feature storage format that reduces storage costs and optimizes off-chip traffic. MEGA [65] employs low precision quantization for non-important vertex features, thus alleviating pressure on DRAM bandwidth. However, since these accelerators are specially designed for traditional GNNs, they are unable to accelerate the compute-intensive Transformer. What's more, the vertex features in traditional GNNs are stored continuously in DRAM, whereas the vertex features in TF-GNNs are irregularly distributed in DRAM due to the random sampling of subgraphs. As a result, these accelerators fail to resolve the bank conflicts within subgraphs as well as capture the locality between subgraphs. In contrast to these approaches, our method takes into account the impact of graph topology on the Transformer and explores the execution patterns of the GNN, uncovering new opportunities arising from the coupling of the structure of the Transformer and GNN.

This paper proposes HEAT, a novel HEterogeneous Architecture for efficient inference of TF-GNN. By deploying the Transformer on a Neural Processing Unit (NPU) and the GNN on a DIMM-based Near-Data Processing (NDP) accelerator, HEAT effectively capitalizes on the high computational power of the NPU and the high internal bandwidth of the NDP, thereby enhancing overall performance. HEAT addresses the following three questions to make our proposal efficient and practical:

1. How to reduce the high computation complexity of the front-end Transformer? Given the observation that vertices with higher degrees are accessed more frequently, such as celebrities commonly mentioned in a social network, the accurate encoding of them is crucial to the final classification results. Based on this insight, we implement a topology-aware encoding algorithm on the front-end Transformer, leveraging the captured importance of vertices to lower the precision of the Transformer that corresponds to the less important vertices. Additionally, we integrate a variable-precision supported Processing Element (PE) in the NPU to facilitate arbitrary bit-width Transformer operations, ensuring that theoretical gains translate into real performance improvements.

2. How to improve the memory efficiency of the back-end GNN? By taking a deep dive into the back-end GNN, we discover that the irregularities within the subgraph can be made more regular through adjustments to both execution granularity and execution order, thus preventing bank conflicts and enhancing locality. First, increasing the execution granularity by bundling multiple subgraphs presents an opportunity to coalesce subgraphs with complementary bank access patterns, thereby reducing bank conflicts. Additionally, reordering the execution order provides a chance to improve locality since neighboring subgraphs often contain frequently accessed *hot vertices*. To fully leverage the locality introduced by hot vertices, we also equip the NDP with a Hot Buffer, which reserves these hot vertices on-chip, thereby minimizing memory overhead.

3. How to leverage the parallelism of NPU and NDP? By studying the TF-GNN dataflow, we find that the dependency between the back-end GNN and the front-end Transformer can be broken by forcing the Transformer to selectively encode vertex features needed by the current subgraph. Decoupling the TF-GNN's dataflow significantly enhances hardware utilization in the heterogeneous architecture, as it allows the NDP to begin back-end GNN computations before the front-end Transformer completes its process.

To better understand the advantages of HEAT, we compare it with previous Transformer and GNN accelerators in Table 1. Compared to existing Transformer accelerators, where they only consider redundant computations within the Transformer, HEAT jointly considers the mutual influence between the Transformer and the GNN, thus adopting varying precisions on the front-end Transformer based on the importance of GNN's vertices. Different from previous GNN accelerators, which only reduce the DRAM bandwidth consumption by compressing the vertex features, HEAT takes into account the impact of the irregular vertex distribution within subgraphs by bundling subgraphs and reordering them, thereby improving the memory efficiency of the back-end GNN. A comprehensive evaluation across multiple datasets demonstrates that HEAT achieves speedups of 20.65×, 11.1×, 10.37×, and 2.36×, as well as energy efficiency improvements of 36.8×, 16.72×, 18.40×, and 2.56×, compared to the high-performance A100 GPU, the Transformer accelerator FACT [40], the GNN accelerator MEGA [65], and a heterogeneous architecture that combines the advantage of FACT and MEGA. The performance improvement of HEAT verifies it as a promising choice for the practical deployment of TF-GNN inference scenarios.

2 Background and Related Work

2.1 Preliminaries

Transformer. A Transformer [51] is typically composed of multiple Transformer layers, each of which contains three key computational stages: QKV projection, attention, and Feed-Forward

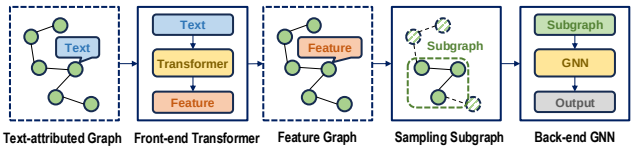

Figure 1: Overview of the common TF-GNN framework.

Network (FFN). Given an input text Token (T), the QKV projection projects it into the Query (Q), Key (K), and Value (V) via General Matrix Multiplication (GEMM): $Q = T \cdot W_Q, K = T \cdot W_K, V = T \cdot W_V$. Next, the attention mechanism calculates the attention Score (S) as follows: $S = softmax(Q \cdot K^T) \cdot V$. Finally, the FFN generates the Output (O) through simple linear layers: $O = W_2(W_1 \cdot S + b_1) + b_2$. Due to the extensive GEMM operations in Transformer layers, Transformers are generally considered to be compute-bound.

Graph Neural Network. A Graph Neural Network (GNN) [26, 45, 52] is typically composed of multiple GNN layers, each of which consists of two primary stages: aggregation and combination. Given an input feature matrix (X), the aggregation stage aggregates the neighboring vertex features via the Adjacency matrix (A), which can be represented as a Sparse Matrix Multiplication (SpMM): $A \cdot X$. Subsequently, the combination stage maps the aggregated feature matrix to a new feature matrix through a Multilayer Perceptron (MLP), which can be expressed as the GEMM of the feature matrix and the Weight matrix (W): $X \cdot W$. Finally, the result is passed through an activation function (σ) to generate the feature matrix for the next layer (X'): $X' = \sigma(A \cdot X \cdot W)$. Due to the extensive memory accesses involved in GNNs, they are typically considered to be memory-bound.

Transformer-empowered GNN. Most Transformer-empowered GNNs (TF-GNNs) [8, 18, 33, 36, 64] follow the framework illustrated in Fig. 1. Given a Text-Attributed Graph (TAG), where each vertex has textual attributes, the front-end Transformer first encodes the text of each vertex into vertex feature, generating a feature graph. This step enhances the representation ability of the vertex features through the Transformer. Subsequently, for each vertex in the feature graph, a K-hop subgraph is sampled with this vertex as the starting point. This step allows a vertex to leverage information from its K-hop neighboring vertices. Finally, each sampled subgraph is fed into the back-end GNN, which aggregates the graph data to generate the final results. Compared to traditional GNNs, TF-GNNs offer up to 30.79% accuracy improvement but come at the cost of more latency and energy consumption [36].

2.2 Hardware Acceleration of Transformers and GNNs

Transformer Accelerators. Currently, there is a series of accelerator works [7, 9, 16, 17, 19, 29, 37, 40, 41, 49, 53, 54, 61] focused on optimizing Transformers, including ELSA [17], Sanger [37], SpAtten [53], and FACT [40]. They generally skip the computation of smaller values in the attention Score S to reduce computational overhead. For example, FACT introduces the Eager Prediction algorithm, which identifies important attention scores early in the Transformer layer and skips the computation of less important values in S. While these works optimize Transformers effectively, they are primarily designed for compute-centric operators (such as attention and FFN) and are not suitable for GNNs, which involve significant memory accesses. Moreover, these works do not account for the specific characteristics of TF-GNNs, such as the varying impact of vertices' degrees on the accuracy of the inference. In contrast, our approach leverages a topology-aware encoding algorithm to determine the precision of the front-end Transformer for each vertex, achieving a better trade-off between accuracy and performance.

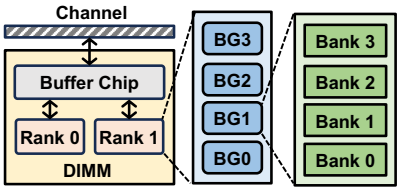

Figure 2: Overview of DRAM hierarchy.

GNN Accelerators. In recent years, a series of GNN accelerators [5, 6, 11, 12, 22, 30, 34, 35, 44, 55, 57, 58, 60, 65] has emerged, including I-GCN [12], SGCN [57], and MEGA [65]. These accelerators aim to alleviate the memory-bound challenges in GNNs by optimizing the storage format of X to reduce memory access overhead. For instance, MEGA observes that the importance of vertex features varies during the combination stage. It further proposes using high precision for important features, while adopting low precision for less important features, thus reducing memory access overhead. However, these accelerators are designed for traditional GNNs, and they are not capable of accelerating the Transformer layers in TF-GNNs, which are highly compute-bound. Moreover, due to the irregular distribution of subgraph vertices in memory within TF-GNNs, it becomes challenging to avoid bank conflicts and improve data locality. Compared to their methods, our approach successfully optimizes the two challenges by exploring both the GNN execution granularity and execution order at the subgraph level.

Near Data Processing based on DRAM. DRAM, as shown in Fig. 2, consists of five hierarchies: channel, DIMM, rank, bank group (BG), and bank. The DRAM channel connects to the host CPU via the memory bus, and typically the bandwidth of a channel is less than the total bandwidth available from all banks. The access latency for a single bank in DRAM is high, and bank conflicts will arise when multiple memory requests are handled by a single bank. Therefore, memory requests are required to be scattered across different banks to exploit parallelism between banks to hide latency. Recently, a series of works [24, 25, 27, 39, 50, 59, 63] have been proposed to reduce the energy consumption of data movement in DRAM through near-data processing (NDP). By placing accelerators on DIMM's buffer chip and exploiting bank- / BG- / rank-level parallelism, DIMM-based NDP can achieve higher memory bandwidth with lower overhead, making it a reasonable choice for accelerating memory-bound GNNs. For example, GraNDe [59] optimizes the arrangement of the input X in DRAM to improve the rank-level parallelism. However, they are still unsuitable for TF-GNN acceleration as their unique model structure and more irregular memory access patterns.

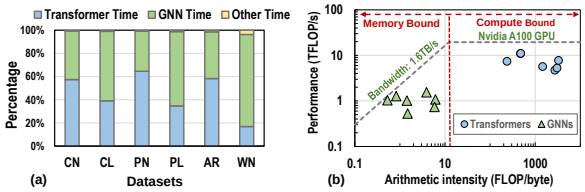

Figure 3: (a) Execution time breakdown of the TF-GNN on different datasets. (b) Roofline model in the TF-GNN.

Table 2: The accuracies of the TF-GNN inference when using ANT [14] framework with different quantization bit-widths on Cora_Node dataset [38].

	FP32	8-bit	7-bit	6-bit	5-bit	4-bit
Accuracy	70.65%	70.50%	69.73%	66.64%	50.39%	37.14%

3 Motivation

To identify performance bottlenecks in the TF-GNN inference, we conduct a breakdown analysis on an A100 GPU. As shown in Fig. 3(a), in most tasks, the front-end Transformer and the back-end GNN dominate the execution time. Therefore, the primary optimization focuses on both the Transformer and GNN. Additionally, to better understand the computation characteristics of the Transformer and GNN, we perform a roofline analysis. Fig. 3(b) illustrates the relationship between their arithmetic intensity (FLOP/byte) and performance (TFLOP/s). We observe that in all datasets, the Transformer is compute-bound, while the GNN is memory-bound. Based on the profiling results, a straightforward approach is to offload the front-end Transformer to an NPU and the back-end GNN to an NDP. Hence, we propose an NPU-NDP heterogeneous architecture to handle the TF-GNN inference. Through a detailed analysis of the TF-GNN, we identify three major challenges and make three corresponding observations.

Challenge 1: ***The computational complexity of the Transformer is excessively high, leading to a significant overhead for the NPU.*** We quantify the computational workload of the Transformer and find that full-precision Transformer requires 41.8 GFLOP in the Cora_Node dataset [38]. To reduce the workload of the Transformer, an intuitive approach is to reduce the model precision through quantization. However, as shown in Table 2, reducing the precision to 5-bit by using the SOTA ANT [14] framework leads to an unacceptable drop in accuracy. The reason behind this is that ANT only considers the raw data distribution in the Transformer, without accounting for the distinct structure of the TF-GNN. This motivates us to explore an adaptive quantization algorithm that can strike a better balance between accuracy and performance.

Observation 1: ***The graph topology implies the importance of input vertex tokens in the Transformer. Therefore, low precision encoding can be applied to less important vertex tokens in the Transformer.*** We find that some vertices in the back-end GNN are less important, and thus can be encoded with lower precision Transformer. To validate this hypothesis, we force the vertex features generated by the Transformer corresponding to the top 10% highest-degree vertices and the bottom 10% lowest-degree vertices to zeros. As shown in Fig. 4, masking the vertex features of the

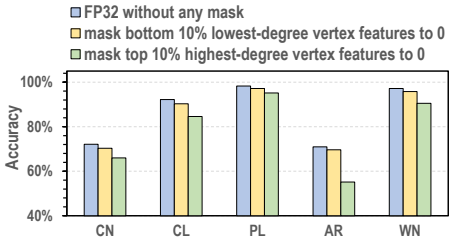

Figure 4: A comparison of accuracy across different datasets using various masking methods.

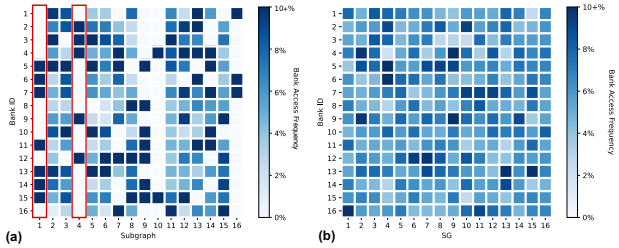

Figure 5: Heatmap of bank access patterns of (a) subgraphs randomly sampled from Cora_Node dataset and (b) Subgraph Groups (SGs) after intra-SG scheduling.

bottom 10% lowest-degree vertices to zero has almost no impact on accuracy, whereas setting the vertex features of the top 10% highest-degree vertices to zero results in a significant accuracy drop (over 5%). This is because the highest-degree vertices appear more frequently in subgraphs and typically have higher importance. Based on the aforementioned findings, we propose a topology-aware encoding algorithm (Section 4), which captures the importance of vertices based on their degree in the topology and applies varying encoding precisions to vertices of different importance, thereby reducing the computational overhead of the Transformer.

Challenges 2: ***The distribution of vertices within the subgraph is irregular, which creates memory pressure for the DIMM-based NDP.*** The irregular vertex distribution within the subgraph presents two main challenges for the DIMM-based NDP: First, there is the issue of bank conflicts during vertex access within the subgraph, which affects the efficiency of DRAM bank access. Second, there are repeated vertices across subgraphs, and reading these vertices multiple times incurs additional memory overhead. Fig. 5(a) visualizes the frequency of DRAM bank accesses using sixteen randomly selected subgraphs, where a darker color means more frequent accesses. It is evident that the subgraph has a bank access pattern clustering in only a few banks, thus encountering severe bank conflicts. As the example in the figure, for Subgraph 1 (S_1), it frequently visits $Bank_5$ several times, but it does not visit $Bank_1$. To better understand redundant memory accesses, we analyze how often each vertex appears in different subgraphs. For example, in the Cora_Node dataset, each vertex will be repeatedly accessed 31 times, leading to significant memory pressure.

Observation 2: ***Subgraphs exhibit bank parallelism and overlapped vertices, which can be leveraged through careful scheduling to optimize the memory overhead of DIMM-based NDP.*** First, we observe that bundling subgraphs gives the flexibility

to balance bank accesses. Considering the first and fourth columns (highlighted in red) in Fig. 5(a), although S_4 rarely accesses $Bank_5$, the access to $Bank_5$ will be complemented by S_1 through bundling them together. This observation allows us to increase the execution granularity by coalescing multiple subgraphs, thus scattering memory accesses across different banks. Fig. 5(b) visualizes the bank access pattern after merging complementary subgraphs. Compared to Fig. 5(a), the accesses are more evenly distributed across different banks, leading to improved bank parallelism. Second, we find that subgraphs sampled from neighboring starting points tend to have more overlapped vertices (referred to as *hot vertices*) than randomly sampled subgraphs. Taking the Cora_Node dataset as an example, subgraphs with neighboring starting points will have 45.81% hot vertices, while for randomly sampled subgraphs, this proportion is only 1.44%. This discrepancy inspires us to change the execution order of subgraphs and reserve hot vertices on-chip to reduce redundant DRAM reads. Based on the two observations, we propose a subgraph scheduling strategy (Section 5.3), which bundles subgraphs aggregating in different banks to reduce bank conflicts (intra-SG scheduling in Section 5.3.1), while also enhancing the proportion of hot vertices through assigning neighboring vertices with higher priority (inter-SG scheduling in Section 5.3.2).

Challenge 3: ***There is an algorithmic dependency between GNN and Transformer, and the parallelism between NPU and NDP is poor.*** Since all vertex features in the subgraph are generated by the Transformer, there is a significant data dependency between the front-end Transformer and the back-end GNN. Since the vertex IDs in the subgraph are irregular, it is difficult to ensure that the required vertex features for the subgraph have already been generated by the Transformer. As a result, the existing dataflow requires that the GNN computation begins only after the Transformer has encoded all the features. This sequential computation between the NPU and DIMM-based NDP reduces their parallelism, increasing the inference latency.

Observation 3: ***A single subgraph does not require all vertex features, and decoupling the dataflow provides opportunities for parallelism.*** Since a subgraph only requires a subset of vertex features at any given time, one straightforward approach is to decouple the original TF-GNN's dataflow, allowing the Transformer to prioritize generating the features needed for the current subgraph. Once all the vertex features for a subgraph are prepared, the GNN computation can begin. Based on the above observation, we propose a decoupled dataflow (Section 5.4), which enhances the parallelism between the NDP and NPU. On the other hand, we observe that subgraphs often have overlapping vertices. To avoid redundant encoding of the same vertices, we introduce an Encode Table (Section 5.4), which enables the NPU to intelligently skip over features that have already been encoded.

4 HEAT Algorithm

We propose a topology-aware encoding algorithm to reduce the expensive computational overhead of the Transformer in TF-GNNs. The innovation of this algorithm lies in utilizing the graph's topology to guide the quantization process applied to the Transformer. The underlying rationale is that vertices with high degrees—referred to as **key vertices**—tend to possess strong representational importance and should therefore be encoded with high precision to preserve accuracy. Conversely, we use a low precision Transformer to encode the remaining vertices (referred to as **non-key vertices**) to reduce computational overhead. In this work, high precision refers to a bit-width range of 8-bit to 10-bit, while low precision corresponds to a range of 2-bit to 7-bit. The detailed methodology for determining the optimal bit-width for TF-GNNs will be discussed in Section 6.1.

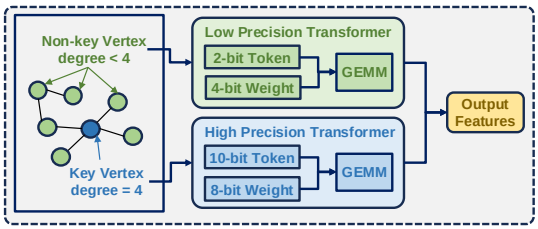

Figure 6: An example of topology-aware encoding.

Algorithm 1: Topology-aware Encoding

input : graph's topology A, percent of key vertices α
output : key vertices K, non-key vertices N
variable : accumulated degree for each vertex D

1 *initial* D;
2 **for** *each edge* (u, v) *in graph* A **do**
3 $D_u \leftarrow D_u + 1$;
4 $D_v \leftarrow D_v + 1$;
5 $VertexIndices, VertexDegrees \leftarrow \text{Sorted}(D)$;
6 **for** $i \leftarrow 0$ **to** $\alpha \cdot |D|$ **do**
7 $K_i \leftarrow VertexIndices_i$;
8 **for** $i \leftarrow 0$ **to** $(1-\alpha) \cdot |D|$ **do**
9 $j \leftarrow i + \alpha \cdot |D|$;
10 $N_i \leftarrow VertexIndices_j$;

The detailed process of our algorithm is described in Algorithm 1. To capture key vertices, we need to use the graph's topology (A) as input. We first traverse each edge of A and count the degree of each vertex (lines 1-4). Then, we reorder the vertices based on their degree in descending order (line 5) and select the top α vertices as key vertices (lines 6-7), with the remaining $1-\alpha$ vertices as non-key vertices (lines 8-10). Here, α is a hyperparameter between 0 and 1; a larger α value provides higher accuracy but also increases the Transformer's workloads. Thus, α needs to be carefully chosen to balance accuracy and computation. We will discuss the selection of α in Section 6.4.2.

After determining which vertices are key vertices and which are non-key vertices, we quantize the input tokens and weight matrices of the Transformer. As shown in Fig. 6, for key vertices (marked in blue), tokens and weights are quantized to 10-bit and 8-bit, respectively. The corresponding computation stages within the Transformer (QKV projection, attention, FFN) are then executed using 10-bit×8-bit operations to preserve inference accuracy. As for

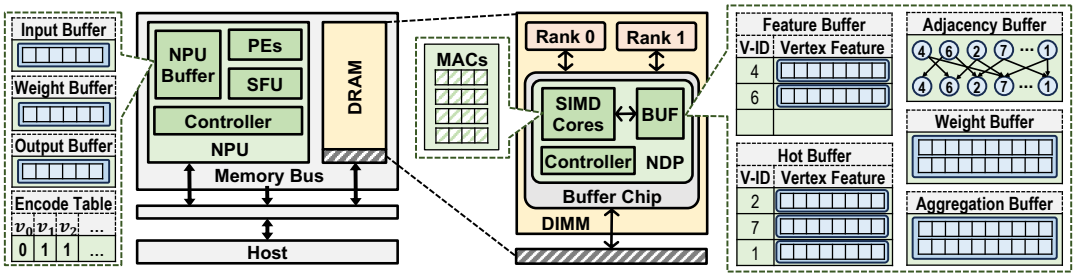

Figure 7: HEAT adopts an NPU-NDP heterogeneous architecture. Green modules are added by HEAT.

non-key vertices (marked in green), tokens and weights are quantized to 2-bit and 4-bit, and all computation stages are performed using 2-bit×4-bit operations to reduce computational cost. Importantly, our approach avoids the need to store multiple sets of weight matrices at different precision levels. For instance, a 4-bit weight matrix can be derived from its 8-bit weight matrix by scaling and clipping, avoiding redundant storage overhead. The quantization formula is:

$$Q_{4bit} = clip(\lfloor \frac{X_{8bit}}{S} \rfloor, 0, 15) \quad (1)$$

where Q_{4bit} represents the quantized 4-bit value, X_{8bit} is the 8-bit value to be quantized, S is the scaling factor, and $clip(X, min, max)$ clips X to the range (min, max).

5 HEAT Architecture

5.1 Overview

To accelerate TF-GNNs, we co-design a hardware accelerator named HEAT. Initially, the host processor prepares the TAG and weight matrices of both the Transformer and GNN in the host memory. HEAT then fetches these data for computation and returns the final result to the host memory. The overall dataflow will be further discussed in Section 5.4.

Fig. 7 shows the overall architecture of HEAT. This architecture adopts a heterogeneous design, consisting of two main modules: the NPU and NDP. The NPU and NDP communicate with the host processor via the memory bus. The NPU contains a PE array, Special Function Units (SFUs), on-chip buffers, and a Controller. The PE array is responsible for the GEMM operations involved in the Transformer, including QKV projection, attention, and FFN. To leverage the theoretical computational benefits brought by the topology-aware encoding discussed in Section 4, the PE array is designed to support flexible bit-width GEMM (Section 5.2.1). To handle the non-linear operations in the Transformer, such as softmax, layer norm, and quantization, we design SFUs to implement these functions. The on-chip buffer in the NPU stores the tokens and weights of the Transformer, while the Controller manages the communication between the NPU, the host, and the DRAM.

The NDP module is located on the DIMM's buffer chip and is shared by all ranks. Our decision is grounded in the following considerations: although locating the NDP within each bank or rank could offer higher bandwidth, it would incur significant overhead for inter-bank/inter-rank communications [13, 21]. The NDP module includes several on-chip buffers, SIMD cores, and a Controller. Its on-chip buffers contain a Feature Buffer storing vertex features,

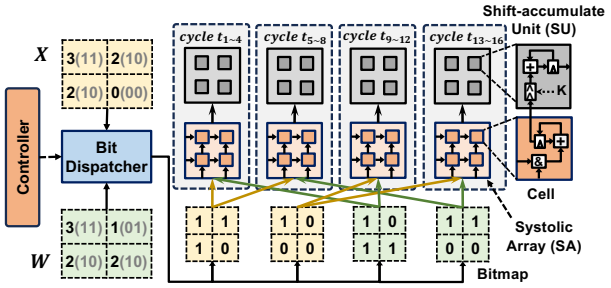

Figure 8: The hardware details of the proposed PE.

an Adjacency Buffer holding the adjacency matrix, a Weight Buffer containing weight matrices, an Aggregation Buffer storing aggregation results, and a Hot Buffer reserving hot vertices to reduce redundant memory accesses (Section 5.3.2). The SIMD cores are responsible for performing GNN's aggregation and combination by integrating multiple Multiply-and-Accumulate (MAC) units. The Controller manages the NDP module's read and write operations to DIMM and coordinates the collaboration of the other sub-modules to complete the full GNN computation.

5.2 Detailed Hardware Design

5.2.1 NPU Design. To support the topology-aware encoding algorithm discussed in Section 4, we incorporate variable-precision supported PEs into the NPU, which enable GEMM operations with arbitrary bit-widths. Unlike the fixed-precision PEs in MEGA and FACT, which fix the bit-width of weights (W) while only allowing variable bit-widths for activations (X), each variable-precision PE is capable of supporting flexible bit-width configurations for both W and X, satisfying the need for efficient execution across a broad range of precision levels. Our key insight is that a GEMM with m-bit W and n-bit X is equivalent to performing GEMM on $m \times n$ 1-bit×1-bit bitmaps. Accordingly, we design the PE shown in Fig. 8, which consists of four main components: a Controller, a Bit Dispatcher, an output-stationary Systolic Array (SA), and 32×32 Shift-accumulate Units (SUs). Each SA consists of 32×32 Cells, with each Cell containing an AND gate, a register and an accumulator. Each cell performs a bitwise AND operation between a 1-bit activation and a 1-bit weight, and accumulates the result with the partial sum stored in the register. Additionally, each Cell is connected to an SU, which is equipped with a shifter, a register and an accumulator. The SU is responsible for shifting the result from its connected Cell by K-bit based on the control signal and accumulating it with the value in the SU register. Specifically, the

Bit Dispatcher routes each bitmap from X and W to SA to perform GEMMs in a sequential order. Take Fig. 8 as an example, given 2×2 W and X, at cycle t_1, the first bitmap of X and the first bitmap of W are fed into the SA. The SA totally takes 3 cycles to complete the GEMM due to the pipeline fill and drain process, after which the controller triggers the SUs to perform shifting and accumulation on the Cells simultaneously. The above process continues four times to consume four bitmap combinations. Please note that the SA and the SUs are organized in a pipelined manner to hide the latency. Although bit-level processing increases the latency of a single PE, we can deploy more PEs under the same area budget to improve overall throughput.

5.2.2 NDP Design. Leveraging the property that the sampled subgraphs are much smaller than the whole graph, we propose accommodating the vertex features of each subgraph on-chip before GNN computation begins. Given that TF-GNNs randomly sample vertices, we introduce a Feature Buffer within the NDP to support efficient indexing of non-sequentially stored vertex features. As shown in Fig. 7, each entry in the Feature Buffer holds two fields: the vertex ID and its corresponding feature vector. When the NDP begins GNN computation, it first fetches the required vertex IDs from DRAM based on the adjacency information stored in the Adjacency Buffer. The SIMD core then compares the incoming vertex IDs with those stored in the Feature Buffer. Upon a match, the corresponding vertex feature is retrieved from the buffer row and used for aggregation. After that, the SIMD core accesses the appropriate weight matrix from the Weight Buffer and performs the feature combination operation. The result is written back to the Feature Buffer as the updated vertex feature for the next GNN layer. This process is repeated iteratively until the GNN computation is complete. For large graphs spanning multiple ranks, we illustrate the NDP's mechanism through the following example. Suppose SG_1 needs to aggregate features of v_1 and v_2, which are located in different ranks, the NDP simultaneously issues read requests to both ranks. As soon as either v_1 or v_2 returns, the NDP stores it in the Feature Buffer and triggers the SIMD Cores to begin accumulating vertex features.

5.3 Subgraph Scheduling Strategy

To optimize memory access efficiency in NDP, we propose a subgraph scheduling strategy that reduces bank conflicts and improves data locality among subgraphs during the offline preprocessing stage. This strategy operates in two phases. In the first phase, referred to as *intra-SG scheduling*, we pack multiple subgraphs into subgraph groups (SGs) so as to scatter their access across different banks, thereby mitigating bank conflicts. In the second phase, termed *inter-SG scheduling*, we change the execution order of SGs to maximize the number of overlapping vertices between consecutive SGs, which improves temporal locality and reduces redundant memory accesses.

5.3.1 Intra-SG Scheduling. To determine which subgraphs should be grouped into an SG, a straightforward approach is to analyze the memory access patterns of all subgraphs and then bundle together those that tend to access the different banks. This ensures that each SG exhibits well-balanced bank access patterns, thereby

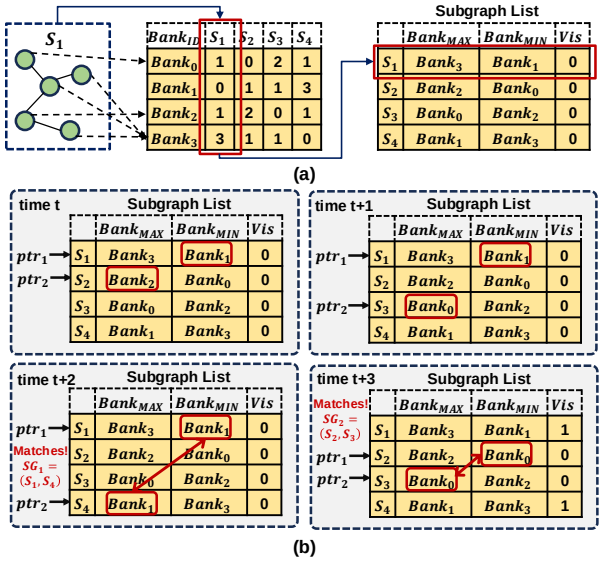

Figure 9: An example of (a) how to record S_1 in Subgraph List, and (b) how intra-SG scheduling bundles subgraphs into SGs using Subgraph List.

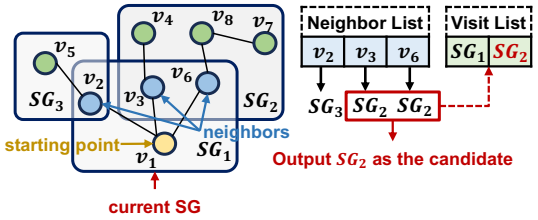

Figure 10: An example of how inter-SG scheduling selects the next candidate SG.

minimizing intra-group bank conflicts. However, implementing this idea is not trivial, as finding the optimal solution by traversing all subgraph combinations is an NP-hard problem. To address this, we propose a greedy scheduling method based on bank access patterns. The basic idea is to merge subgraphs with complementary access characteristics: if a subgraph exhibits sparse access to a particular bank, it is paired with another subgraph that frequently accesses that bank. Overall, the algorithm consists of two stages: the first stage calculates the bank access pattern, and the second stage uses a greedy algorithm to obtain the SGs.

In the first stage, considering that vertex features of subgraphs are stored contiguously in DRAM, we can obtain the distribution of vertex features by calculating the offset of addresses. Specifically, given the vertex ID (V_{ID}), feature dimensions (Dim), feature bitwidth (Bit), DRAM row size (Row_{size}), and the number of DRAM banks ($Bank_{num}$), we can calculate the bank ID ($Bank_{ID}$) corresponding to each vertex feature under bank interleaving by:

$$Bank_{ID} = \left\lfloor \frac{V_{ID} \times Dim \times Bit}{Row_{size}} \right\rfloor \bmod Bank_{num} \tag{2}$$

Moreover, given the number of rows per bank (Row_{num}), we can similarly compute the $Bank_{ID}$ of each vertex feature under row interleaving:

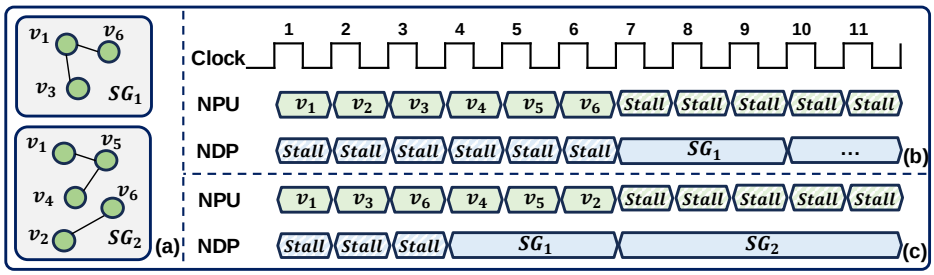

Figure 11: (a) A TF-GNN inference task with two SGs. (b) Naive dataflow. (c) Proposed decoupled dataflow.

$$Bank_{ID} = \left\lfloor \frac{V_{ID} \times Dim \times Bit}{Row_{size} \times Row_{num}} \right\rfloor \quad (3)$$

In the second stage, our algorithm identifies the most frequently accessed bank ($Bank_{MAX}$) and the least frequently accessed bank ($Bank_{MIN}$) for each subgraph. If the $Bank_{MAX}$ of one subgraph matches the $Bank_{MIN}$ of another subgraph, their combined bank access pattern will be more balanced. We record the $Bank_{ID}$ of each subgraph's $Bank_{MAX}$ and $Bank_{MIN}$ sequentially in a Subgraph List, as shown in Fig. 9(a). For S_1, it accesses $Bank_3$ three times and never accesses $Bank_1$. Therefore, in the first row in Subgraph List, we record $Bank_{MAX}$ as $Bank_3$ and $Bank_{MIN}$ as $Bank_1$. To implement the greedy scheduling method, we use two pointers, ptr_1 and ptr_2, which point to the current subgraph and a candidate subgraph waiting to be merged in the Subgraph List, respectively. In Fig. 9(b), when the algorithm begins, ptr_1 and ptr_2 are initialized to point to the first and second entries in the Subgraph List. For the subgraph pointed by ptr_1, we compare its $Bank_{MIN}$ with the $Bank_{MAX}$ of the subgraph pointed by ptr_2. If their $Bank_{ID}$ matches, they are bundled into an SG; otherwise, ptr_2 is moved to the next subgraph, and the comparison is repeated. In the figure, at the time t+2, the $Bank_{MIN}$ of S_1 matches the $Bank_{MAX}$ of S_4, so S_1 and S_4 are bundled into SG_1. Then, at time t+3, ptr_1 points to S_2, ptr_2 points to S_3, and it is again found that S_2 and S_3 match successfully. Therefore, S_2 and S_3 are bundled into SG_2.

Additionally, to prevent a subgraph from repeatedly appearing in multiple SGs, we introduce a valid bit (Vis) in the Subgraph List to track whether the subgraph has already been assigned to another SG. Both ptr_1 and ptr_2 continue to point to rows whose Vis is 0 (False) until all subgraphs are assigned to SGs.

5.3.2 Inter-SG Scheduling and Hot Buffer Design. After obtaining the SGs through intra-SG scheduling, the next step is to determine the execution order of these SGs, with the primary goal being maximizing the number of hot vertices between SGs. We observe that when the starting point of the next subgraph is adjacent to that of the current subgraph, a higher degree of overlap typically occurs. Based on this observation, we traverse the neighbors within the current SG and select the next execution SG with the highest count of overlapping vertices.

As illustrated in Fig. 10, we begin by maintaining a Neighbor List that records the neighbors of the current SG SG_1. For each vertex in this list, we determine its corresponding SG and count the number of overlapping vertices between other SGs and SG_1. In the example shown, SG_2 is selected as the next execution target because it exhibits the highest number of overlapping vertices with SG_1. Moreover, to avoid selecting an already visited SG, we also use a Visit List to track whether an SG has been accessed. This process iterates until all SGs have been assigned an access order.

Compared to a random SG execution order, the reordered SGs through inter-SG scheduling contain a higher proportion of hot vertices. To avoid redundant DRAM accesses and improve data reuse, we reserve the features of these hot vertices in on-chip memory. To this end, we co-design a hardware component called the Hot Buffer, which temporarily stores the features of hot vertices shared between SGs. To determine which vertices should be stored in the Hot Buffer, we record the overlapping vertices between SGs during the offline SG generation and mark these vertices' IDs as hot IDs. As shown in Fig. 7, each entry in the Hot Buffer stores a hot vertex ID along with its corresponding feature. During GNN computation, the NDP Controller checks whether the ID of a fetched vertex matches any entry in the Hot Buffer. If a match (i.e., a hit) is found, the vertex feature is directly retrieved from the Hot Buffer. Otherwise, the NDP fetches the vertex feature from DRAM and stores it in the Feature Buffer for subsequent use.

5.4 Decoupled Dataflow

As discussed in Section 2.1, the naive dataflow of TF-GNNs follows a sequential execution mode. Specifically, vertex features are first encoded by the Transformer on the NPU. Once all vertex features have been encoded, the SGs and hot IDs, generated by the subgraph scheduling strategy mentioned in Section 5.3, are transferred to the NDP to initiate GNN computation. However, the NDP encounters severe hardware under-utilization when the NPU encodes vertex features. Therefore, we propose a decoupled dataflow that breaks the inherent dependency of the Transformer and the GNN. It forces the Transformer to directly generate the vertex features required for the SG so that the GNN computation can start immediately after the required vertex features are ready. This approach enables pipelined parallelism between the NPU and NDP. Fig. 11(b)-(c) shows a comparison between the naive dataflow and the decoupled dataflow. In the naive dataflow, the GNN computation for SG_1 in the NDP cannot begin until all the vertices from v_1 to v_6 are generated in the NPU, resulting in 6 cycles of idle time. In contrast, the decoupled dataflow allows the NPU to begin computation for SG_1 with vertices v_1, v_3, and v_6 first. After the computation of the vertices in SG_1 is completed, the NDP can begin the GNN computation for SG_1 without delay, reducing the idle time by 3 cycles.

To avoid re-encoding the same vertex features across different SGs, the NPU maintains an Encode Table that records whether a

Table 3: The statistics of datasets used in this work.

Dataset	Task	#Nodes	#Edges	#Classes
Cora_Node (CN) [38]	Node	2708	10556	7
Cora_Link (CL) [38]	Link	2708	10556	7
PubMed_Node (PN) [47]	Node	19717	44338	3
PubMed_Link (PL) [47]	Link	19717	44338	3
ARxiv (AR) [20]	Node	169343	1166243	40
WN18RR (WN) [3]	Link	40943	93003	11

Table 4: The configuration and breakdown of HEAT.

	Component	Area(mm^2)	Power(mW)	Config
NPU	PE Array	0.88	200.70	32 × PE
	SFUs	0.28	11.98	-
	NPU Buffer	1.28	114.25	512KB
Total of NPU		2.44(65.95%)	326.93(70.57%)	-
NDP	SIMD Cores	0.13	38.69	32 × 16-way
	Adjacency Buffer	0.09	3.34	12KB
	Weight Buffer	0.15	11.44	48KB
	Feature Buffer	0.36	31.60	128KB
	Aggregation Buffer	0.29	29.16	128KB
	Hot Buffer	0.24	22.08	96KB
Total of NDP		1.26(34.05%)	136.31(29.43%)	-
Total of HEAT (28nm)		3.70	463.24	-
DRAM Timing Parameter				
tRP = 22, tRCD = 22, tRAS = 52, tCCD_S = 4, tCCD_L = 8, tREFI = 12480, tRFC = 560				

vertex has been encoded. When a new vertex is sent from the host to the NPU, the NPU's Controller first accesses the Encode Table. If the entry in the Encode Table is already True, the NPU skips the encoding for that vertex. Otherwise, the NPU triggers the PEs for Transformer encoding and updates the associated entry.

6 Evaluation

6.1 Experiments Methodology

Models and Datasets. We follow the SOTA TF-GNN frameworks [8, 18, 33, 36, 64], utilizing SentenceBert [42] as the front-end Transformer model, and RGCN [46] and RGAT [4] as the back-end GNN models with 2-hop subgraph sampling. For each model, we use the recommended parameter settings from the open-source code[1]. In our evaluation, we use six datasets, including three node prediction datasets (Cora_Node (CN), PubMed_Node (PN), Arxiv (AR)) and three link prediction datasets (Cora_Link (CL), PubMed_Link (PL), WN18RR (WN)). Details about these datasets are shown in Table 3.

Experiment Setup. To model the behavior of HEAT, we build a cycle-accurate simulator written in C++. To support HEAT's heterogeneous architecture, we develop an NPU module based on the systolic array design from ONNXim [15] and implement a near-rank NDP module based on a DDR4_8Gb_x4_3200 module in DRAMsim3 [31]. We implement and synthesize the digital logic circuits for each module at the 28nm technology node with 500MHz using Synopsys Design Compiler to obtain area and power consumption. The on-chip buffers are evaluated using CACTI 7.0 [2] with a 32nm technology node, and we use the DeepScaleTool [43] to scale the results down to 28nm. For GPU comparison, we use an A100 GPU with 80GB VRAM as the baseline. To compare HEAT with previous accelerators, we select the SOTA Transformer accelerator FACT [40] and GNN accelerator MEGA [65] as baselines. Since they are specially designed for Transformers and GNNs, we assign FACT to handle the Transformer and MEGA to handle the GNN. More specifically, we adopt three configurations:

(i) FACT handles the Transformer, and the GPU handles the GNN (FACT).
(ii) The GPU handles the Transformer, and MEGA handles the GNN (MEGA).
(iii) FACT handles the Transformer, and MEGA handles the GNN (FACT+MEGA).

To make a fair comparison, we use the same DDR4_8Gb_x4_3200 module for FACT and MEGA, and scale their compute units and buffers under the same area budget as HEAT.

[1]https://github.com/LechengKong/OneForAll

Table 5: Comparison of the time needed by HEAT's offline algorithms with the training time of TF-GNN.

Time (s)	CN	CL	PN	PL	AR	WM
Topology-aware Encoding	0.04	0.04	0.18	0.18	0.98	0.23
Intra-SG Scheduling	0.16	0.32	1.14	3.94	14.92	7.53
Inter-SG Scheduling	0.11	0.28	1.02	4.39	16.45	7.26
TF-GNN Training	112.57	646.90	109.74	2205.87	12028.22	1693.15

Algorithm Details. In Section 4, we leave the determination of the high/low precision bit-widths in the HEAT algorithm unexplained. Here we will elaborate on how we determine those bit-widths. Following [40, 48, 62], we employ the successive halving (SH) method during training to determine the optimal bit-widths for the Transformer based on the training loss. Note that training loss is affected by both training accuracy and Transformer precision, and it varies for each dataset. Specifically, we start training on all possible bit-width groups. In each epoch, half of the groups with the highest training loss are discarded, and this process iterates until only one group remains. This remaining bit-width group is then applied to the HEAT algorithm as the precision for the Transformer.

6.2 Algorithm Results

6.2.1 Accuracy. We first evaluate the accuracy impact of the HEAT algorithm on TF-GNNs. To better understand HEAT's advantage, we select the SOTA ANT framework for comparison. When using ANT for quantization, we select three precision levels: 8-bit, 6-bit, and 4-bit, which are referred to as ANT 8-bit, ANT 6-bit, and ANT 4-bit, respectively. Note that both HEAT and ANT perform quantization only on the front-end Transformer, while the back-end GNN remains 8-bit precision. Fig. 12 shows the accuracy of HEAT compared with ANT. Extensive experiments demonstrate that HEAT incurs almost no accuracy loss compared to ANT 8-bit, achieves slightly higher accuracy than ANT 6-bit, and exhibits significantly higher accuracy compared to ANT 4-bit. Meanwhile, we also compared their normalized computation of the Transformer in Fig. 13. Compared to ANT 6-bit in both figures, HEAT not only achieves higher accuracy but also has a smaller normalized computation than ANT 6-bit. This is because ANT only considers the raw data distribution of the Transformer, whereas the HEAT algorithm

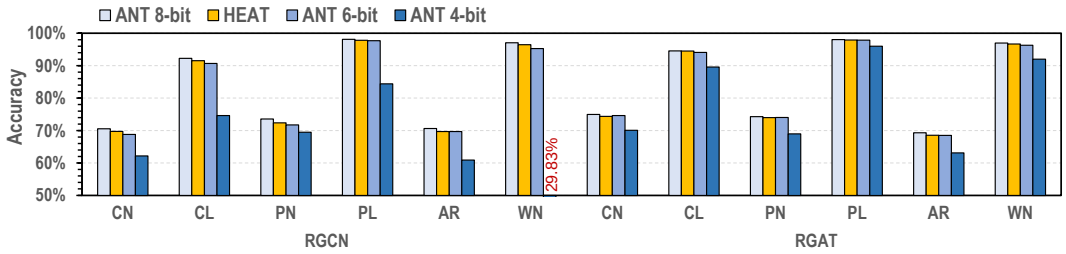

Figure 12: Accuracy of HEAT comparing with ANT framework on different datasets utilizing both RGCN and RGAT.

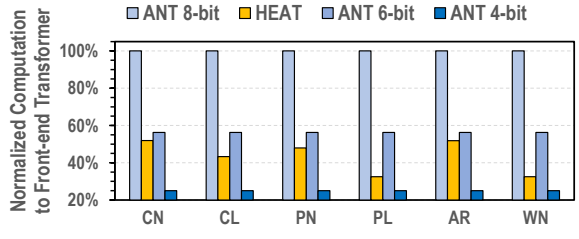

Figure 13: A comparison of the computation workload of the front-end Transformer, the results are normalized to ANT 8-bit. Despite the significant accuracy drop of ANT 4-bit, we still include it in our comparison for a better understanding.

leverages the topology to capture the importance of vertex features, allowing for a better trade-off between accuracy and performance.

We also observe that ANT 4-bit exhibits significant performance variation across different datasets, which is a phenomenon caused by both the model architecture and the task complexity. For models with larger parameters (e.g., RGAT) or relatively simpler tasks (e.g., PL), the accuracy degradation under 4-bit quantization is notably smaller, as the inherent robustness of the model helps to mitigate the errors introduced by low precision quantization.

6.2.2 Algorithm Cost. To better understand the cost of algorithms in HEAT, Table 5 presents the time required by each algorithm across different datasets. It can be observed that the time needed by topology-aware encoding and intra-/inter-SG scheduling increases with the size of dataset, but remains negligible compared to the training time of TF-GNN.

6.3 Hardware Results

6.3.1 Speedup. As shown in Fig. 14, HEAT achieves an average speedup of 20.65×, 11.1×, 10.37×, and 2.36× compared to GPU, FACT, MEGA, and FACT+MEGA, respectively. The speedup over the GPU is significant because HEAT adopts a specialized architecture, which uses variable-precision supported PEs to reduce computational overhead in the front-end Transformer and adopts a DIMM-based NDP design to improve memory efficiency in the back-end GNN. Compared to FACT, HEAT's performance improvement comes from two main reasons. First, FACT is designed for the front-end Transformer, so it cannot accelerate the memory-bound back-end GNN. For example, on the memory-intensive dataset WN18RR in RGCN, HEAT achieves a great 21.81× speedup over FACT. Second, HEAT leverages the graph's topology in the back-end GNN to capture less important vertices and encodes their features with the low precision Transformer, which further optimizes the cost of the front-end Transformer. As for MEGA, HEAT also outperforms it for two factors. The first factor is similar to that for FACT: MEGA is designed to accelerate the back-end GNN, so it cannot alleviate the substantial computational overhead in the front-end Transformer. The second factor is that HEAT further exploits new opportunities brought by subgraphs in the back-end Transformer. By bundling and rearranging subgraphs, HEAT significantly reduces bank conflicts in DRAM and improves the locality between subgraphs, thereby minimizing redundant memory accesses. Finally, HEAT still outperforms the FACT+MEGA architecture. This is because HEAT leverages decoupled dataflow to improve the hardware utilization of both NPU and NDP, whereas FACT+MEGA are constrained by the inherent algorithmic dependencies in TF-GNN, causing MEGA's hardware to remain idle during FACT's computation. In summary, neither the FACT nor MEGA can fully unleash their power in the new scenario of TF-GNNs. In contrast, HEAT leverages the new opportunities brought by the coupling of the front-end Transformer and back-end GNN, thus achieving significant performance gains and demonstrating its potential value in real-world TF-GNN deployments.

6.3.2 Energy Efficiency. Fig. 15 illustrates the energy efficiency of HEAT and baselines. HEAT offers an average energy efficiency of 36.8×, 16.72×, 18.40×, and 2.56× compared to GPU, FACT, MEGA, and FACT+MEGA, respectively. To better understand the sources of energy savings, we present a breakdown analysis of energy consumption in Fig. 16. HEAT's energy benefits stem from reductions in both computation energy and memory energy. The savings in computation energy arise from the decrease in the computation workload of the front-end Transformer (see Fig. 13), and we employ variable-precision supported PEs to translate theoretical gains into practical benefits. Additionally, through inter-SG scheduling and reserving hot vertices in the Hot Buffer, we significantly reduce redundant DRAM accesses, which further lowers energy consumption. It is important to note that when compared to FACT+MEGA, HEAT still achieves significant energy efficiency because HEAT uses near-data processing, which reduces the energy consumption associated with accessing data from banks.

6.3.3 Area and Power. The area and power consumption of HEAT are summarized in Table 4. The NPU accounts for the majority of power consumption, as the front-end Transformer is a compute-centric task that requires substantial computational resources.

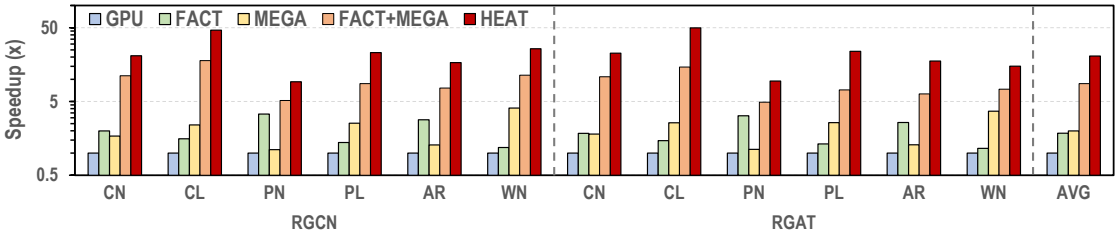

Figure 14: Speedup of HEAT comparing with GPU, FACT, MEGA, and FACT+MEGA on different datasets utilizing RGCN and RGAT, the results are normalized to GPU.

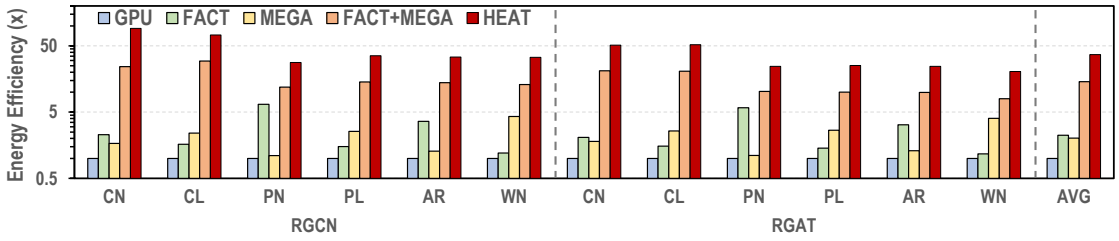

Figure 15: Energy efficiency of HEAT comparing with GPU, FACT, MEGA, and FACT+MEGA on different datasets utilizing RGCN and RGAT, the results are normalized to GPU.

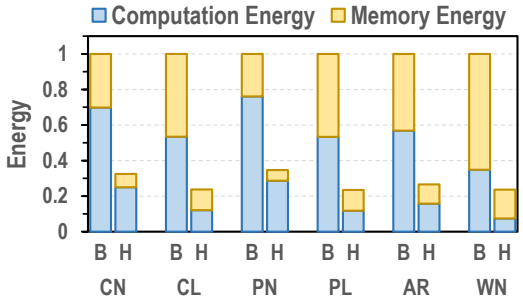

Figure 16: Normalized energy consumption of HEAT (H) comparing with Base NPU-NDP architecture (B) on different datasets utilizing RGCN.

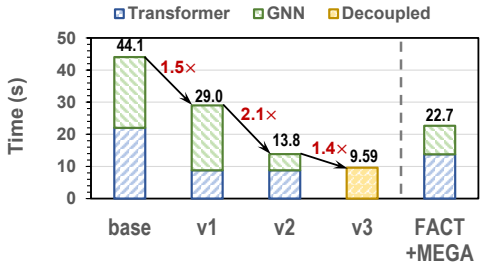

Figure 17: The contribution of each method to the saving of execution time. We also include the performance of FACT+MEGA to provide a more comprehensive comparison.

Moreover, the area of the NDP module is 1.26 mm^2, which corresponds to nearly 1.3% of the buffer chip in the DIMM [1]. Also, the power of the NDP module is 136.31 mW, which is also negligible compared to that in the buffer chip. Therefore, such an NDP module can be easily integrated into the commodity DRAM design.

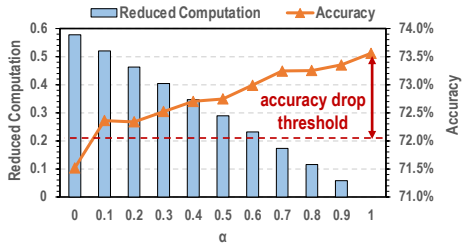

Figure 18: Reduced computation in the Transformer (higher is better) and the TF-GNN's inference accuracy (higher is better) under different α on PubMed_Node dataset with RGCN.

6.4 Detailed Results

6.4.1 Ablation Study. To analyze where our gains come from and demonstrate the effectiveness of our proposed techniques, we conduct an ablation study on execution time for both Transformer and GNN. Fig. 17 visualizes the contribution of each proposed method: naive NPU-NDP architecture (base), base with topology-aware encoding (v1), v1 with subgraph scheduling strategy (v2), and v2 with decoupled dataflow (v3). Starting from base to v1, the topology-aware encoding effectively reduces the overhead of the front-end Transformer, resulting in a 1.5× speedup. However, at this stage, the back-end GNN becomes the primary bottleneck. Therefore, from v1 to v2, we introduce a subgraph scheduling strategy to reduce bank conflicts and improve locality, which significantly alleviates the memory pressure in the back-end GNN, further providing a 2.1× speedup. Finally, by decoupling the dataflow, we increased the parallelism between the front-end Transformer and back-end GNN,

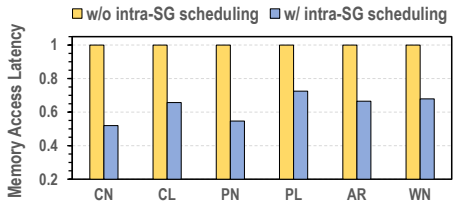

Figure 19: The effect of intra-SG scheduling in memory access latency (lower is better) under different datasets using RGCN.

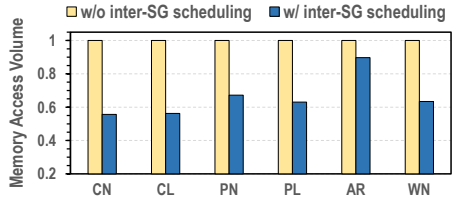

Figure 20: The effect of inter-SG scheduling in memory access volume (lower is better) under different datasets using RGCN.

leading to an additional 1.4× speedup. Moreover, we visualize the performance of FACT+MEGA and observe that v2 outperforms it in both the Transformer and GNN components, thereby demonstrating the superiority of HEAT.

6.4.2 Parameter Trade-off. In HEAT, we need to carefully determine the hyperparameter α in the topology-aware encoding, as it simultaneously affects both the TF-GNN's accuracy and the reduction in computation workloads. To find the optimal α, we use the Pubmed_Node dataset as an example and vary α from 0 to 1 to explore a good balance between reduced computation in the Transformer and model accuracy. In Fig. 18, we observe that the increasing α comes with a higher accuracy due to more vertices being selected as key vertices. Moreover, reduced computation increases as α decreases, which benefits from more vertex features being encoded with the low precision Transformer. Therefore, to maximize reduced computation while maintaining high accuracy, we choose 0.1 as the α since it achieves the maximum computation reduction while staying within a 1.5% accuracy loss threshold.

6.4.3 Impact of Subgraph Scheduling Strategy. To better understand the proposed memory-efficient subgraph scheduling strategy mentioned in Section 5.3, we will provide a more detailed discussion of intra-SG scheduling and inter-SG scheduling below.

Intra-SG scheduling. Intra-SG scheduling reduces bank conflicts by bundling subgraphs with complementary bank access patterns into a single SG, thereby reducing memory access latency. Fig. 19 shows the changes in memory access latency after applying intra-SG scheduling across different datasets in RGCN. As seen in the figure, intra-SG scheduling reduces the average memory access latency by 36.79%. This is primarily due to its effectiveness in reducing bank conflicts, which in turn allows for better utilization of multi-bank parallelism to hide latency.

Inter-SG scheduling. Inter-SG scheduling adjusts the execution order of SGs by assigning neighboring SGs with higher priority, thereby increasing the number of hot vertices between adjacent SGs and reducing redundant DRAM accesses. In Fig. 20, we show the changes in DRAM accesses across different datasets for RGCN after applying inter-SG scheduling. It is evident that inter-SG scheduling effectively reduces redundant memory accesses, with an average reduction of 34.13%.

By combining intra-SG scheduling and inter-SG scheduling, we reduce memory access latency as well as decrease the overall volume of memory accesses, resulting in significant performance gains.

7 Conclusion

In this paper, we propose HEAT, a novel heterogeneous architecture designed for efficient TF-GNN inference. By deploying the Transformer on the NPU and the GNN on a DIMM-based NDP, HEAT harnesses both the high computational throughput of the NPU and the high internal bandwidth of the NDP. HEAT first leverages graph topology to measure vertex importance and applies importance-aware quantization to the Transformer. Furthermore, it introduces a flexible subgraph scheduling mechanism that reorganizes both the execution granularity and order of GNN subgraphs, effectively reducing bank conflicts and enhancing data locality. Finally, HEAT decouples the inherent dependency between the front-end Transformer and back-end GNN, enabling their parallel execution on NPU and NDP. Extensive experiments show that HEAT delivers significant performance gains without compromising model accuracy.

Acknowledgments

We thank the anonymous reviewers for their valuable feedback. This work is partly supported by the National Natural Science Foundation of China (Grant No. 62202288).

References

[1] Mohammad Alian, Seung Won Min, Hadi Asgharimoghaddam, Ashutosh Dhar, Dong Kai Wang, Thomas Roewer, Adam McPadden, Oliver O'Halloran, Deming Chen, Jinjun Xiong, Daehoon Kim, Wen-mei Hwu, and Nam Sung Kim. 2018. Application-Transparent Near-Memory Processing Architecture with Memory Channel Network. In *2018 51st Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. 802–814. https://doi.org/10.1109/MICRO.2018.00070

[2] Rajeev Balasubramonian, Andrew B. Kahng, Naveen Muralimanohar, Ali Shafiee, and Vaishnav Srinivas. 2017. CACTI 7: New Tools for Interconnect Exploration in Innovative Off-Chip Memories. *ACM Trans. Archit. Code Optim.* 14, 2, Article 14 (June 2017), 25 pages. https://doi.org/10.1145/3085572

[3] Antoine Bordes, Nicolas Usunier, Alberto Garcia-Duran, Jason Weston, and Oksana Yakhnenko. 2013. Translating Embeddings for Modeling Multi-relational Data. In *Advances in Neural Information Processing Systems*, C.J. Burges, L. Bottou, M. Welling, Z. Ghahramani, and K.Q. Weinberger (Eds.), Vol. 26. Curran Associates, Inc. https://proceedings.neurips.cc/paper_files/paper/2013/file/1cecc7a77928ca8133fa24680a88d2f9-Paper.pdf

[4] Dan Busbridge, Dane Sherburn, Pietro Cavallo, and Nils Y Hammerla. 2019. Relational graph attention networks. *arXiv preprint arXiv:1904.05811* (2019).

[5] Cen Chen, Kenli Li, Yangfan Li, and Xiaofeng Zou. 2022. ReGNN: A Redundancy-Eliminated Graph Neural Networks Accelerator. In *2022 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. 429–443. https://doi.org/10.1109/HPCA53966.2022.00039

[6] Cen Chen, Kenli Li, Xiaofeng Zou, and Yangfan Li. 2021. DyGNN: Algorithm and Architecture Support of Dynamic Pruning for Graph Neural Networks. In *2021 58th ACM/IEEE Design Automation Conference (DAC)*. 1201–1206. https://doi.org/10.1109/DAC18074.2021.9586298

[7] Hongzheng Chen, Jiahao Zhang, Yixiao Du, Shaojie Xiang, Zichao Yue, Niansong Zhang, Yaohui Cai, and Zhiru Zhang. 2024. Understanding the Potential of FPGA-based Spatial Acceleration for Large Language Model Inference. *ACM Trans. Reconfigurable Technol. Syst.* 18, 1, Article 5 (Dec. 2024), 29 pages. https://doi.org/10.1145/3656177

[8] Eli Chien, Wei-Cheng Chang, Cho-Jui Hsieh, Hsiang-Fu Yu, Jiong Zhang, Olgica Milenkovic, and Inderjit S Dhillon. 2021. Node feature extraction by self-supervised multi-scale neighborhood prediction. *arXiv preprint arXiv:2111.00064* (2021).

[9] Hongxiang Fan, Thomas Chau, Stylianos I. Venieris, Royson Lee, Alexandros Kouris, Wayne Luk, Nicholas D. Lane, and Mohamed S. Abdelfattah. 2022. Adaptable Butterfly Accelerator for Attention-based NNs via Hardware and Algorithm Co-design. In *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. 599–615. https://doi.org/10.1109/MICRO56248.2022.00050

[10] Wenqi Fan, Yao Ma, Qing Li, Yuan He, Eric Zhao, Jiliang Tang, and Dawei Yin. 2019. Graph Neural Networks for Social Recommendation. In *The World Wide Web Conference* (San Francisco, CA, USA) *(WWW '19)*. Association for Computing Machinery, New York, NY, USA, 417–426. https://doi.org/10.1145/3308558.3313488

[11] Tong Geng, Ang Li, Runbin Shi, Chunshu Wu, Tianqi Wang, Yanfei Li, Pouya Haghi, Antonino Tumeo, Shuai Che, Steve Reinhardt, and Martin C. Herbordt. 2020. AWB-GCN: A Graph Convolutional Network Accelerator with Runtime Workload Rebalancing. In *2020 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. 922–936. https://doi.org/10.1109/MICRO50266.2020.00079

[12] Tong Geng, Chunshu Wu, Yongan Zhang, Cheng Tan, Chenhao Xie, Haoran You, Martin Herbordt, Yingyan Lin, and Ang Li. 2021. I-GCN: A Graph Convolutional Network Accelerator with Runtime Locality Enhancement through Islandization. In *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture* (Virtual Event, Greece) *(MICRO '21)*. Association for Computing Machinery, New York, NY, USA, 1051–1063. https://doi.org/10.1145/3466752.3480113

[13] Christina Giannoula, Peiming Yang, Ivan Fernandez, Jiacheng Yang, Sankeerth Durvasula, Yu Xin Li, Mohammad Sadrosadati, Juan Gomez Luna, Onur Mutlu, and Gennady Pekhimenko. 2025. PyGim: An Efficient Graph Neural Network Library for Real Processing-In-Memory Architectures. arXiv:2402.16731 [cs.AR] https://arxiv.org/abs/2402.16731

[14] Cong Guo, Chen Zhang, Jingwen Leng, Zihan Liu, Fan Yang, Yunxin Liu, Minyi Guo, and Yuhao Zhu. 2022. ANT: Exploiting Adaptive Numerical Data Type for Low-bit Deep Neural Network Quantization. In *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. 1414–1433. https://doi.org/10.1109/MICRO56248.2022.00095

[15] Hyungkyu Ham, Wonhyuk Yang, Yunseon Shin, Okkyun Woo, Guseul Heo, Sangyeop Lee, Jongse Park, and Gwangsun Kim. 2024. ONNXim: A Fast, Cycle-Level Multi-Core NPU Simulator. *IEEE Computer Architecture Letters* 23, 2 (2024), 219–222. https://doi.org/10.1109/LCA.2024.3484648

[16] Tae Jun Ham, Sung Jun Jung, Seonghak Kim, Young H. Oh, Yeonhong Park, Yoonho Song, Jung-Hun Park, Sanghee Lee, Kyoung Park, Jae W. Lee, and Deog-Kyoon Jeong. 2020. A^3: Accelerating Attention Mechanisms in Neural Networks with Approximation. In *2020 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. 328–341. https://doi.org/10.1109/HPCA47549.2020.00035

[17] Tae Jun Ham, Yejin Lee, Seong Hoon Seo, Soosung Kim, Hyunji Choi, Sung Jun Jung, and Jae W. Lee. 2021. ELSA: Hardware-Software Co-design for Efficient, Lightweight Self-Attention Mechanism in Neural Networks. In *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*. 692–705. https://doi.org/10.1109/ISCA52012.2021.00060

[18] Xiaoxin He, Xavier Bresson, Thomas Laurent, Adam Perold, Yann LeCun, and Bryan Hooi. 2023. Harnessing explanations: Llm-to-lm interpreter for enhanced text-attributed graph representation learning. *arXiv preprint arXiv:2305.19523* (2023).

[19] Seongmin Hong, Seungjae Moon, Junsoo Kim, Sungjae Lee, Minsub Kim, Dongsoo Lee, and Joo-Young Kim. 2022. DFX: A Low-latency Multi-FPGA Appliance for Accelerating Transformer-based Text Generation. In *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. 616–630. https://doi.org/10.1109/MICRO56248.2022.00051

[20] Weihua Hu, Matthias Fey, Marinka Zitnik, Yuxiao Dong, Hongyu Ren, Bowen Liu, Michele Catasta, and Jure Leskovec. 2021. Open Graph Benchmark: Datasets for Machine Learning on Graphs. arXiv:2005.00687 [cs.LG] https://arxiv.org/abs/2005.00687

[21] Wenqin Huangfu, Xueqi Li, Shuangchen Li, Xing Hu, Peng Gu, and Yuan Xie. 2019. MEDAL: Scalable DIMM based Near Data Processing Accelerator for DNA Seeding Algorithm. In *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture* (Columbus, OH, USA) *(MICRO '52)*. Association for Computing Machinery, New York, NY, USA, 587–599. https://doi.org/10.1145/3352460.3358329

[22] Ranggi Hwang, Minhoo Kang, Jiwon Lee, Dongyun Kam, Youngjoo Lee, and Minsoo Rhu. 2023. GROW: A Row-Stationary Sparse-Dense GEMM Accelerator for Memory-Efficient Graph Convolutional Neural Networks. In *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. 42–55. https://doi.org/10.1109/HPCA56546.2023.10070983

[23] Bowen Jin, Chen Gao, Xiangnan He, Depeng Jin, and Yong Li. 2020. Multi-behavior Recommendation with Graph Convolutional Networks. In *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval* (Virtual Event, China) *(SIGIR '20)*. Association for Computing Machinery, New York, NY, USA, 659–668. https://doi.org/10.1145/3397271.3401072

[24] Liu Ke, Udit Gupta, Benjamin Youngjae Cho, David Brooks, Vikas Chandra, Utku Diril, Amin Firoozshahian, Kim Hazelwood, Bill Jia, Hsien-Hsin S. Lee, Meng Li, Bert Maher, Dheevatsa Mudigere, Maxim Naumov, Martin Schatz, Mikhail Smelyanskiy, Xiaodong Wang, Brandon Reagen, Carole-Jean Wu, Mark Hempstead, and Xuan Zhang. 2020. RecNMP: Accelerating Personalized Recommendation with Near-Memory Processing. In *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*. 790–803. https://doi.org/10.1109/ISCA45697.2020.00070

[25] Liu Ke, Xuan Zhang, Jinin So, Jong-Geon Lee, Shin-Haeng Kang, Sukhan Lee, Songyi Han, YeonGon Cho, Jin Hyun Kim, Yongsuk Kwon, KyungSoo Kim, Jin Jung, Ilkwon Yun, Sung Joo Park, Hyunsun Park, Joonho Song, Jeonghyeon Cho, Kyomin Sohn, Nam Sung Kim, and Hsien-Hsin S. Lee. 2022. Near-Memory Processing in Action: Accelerating Personalized Recommendation With AxDIMM. *IEEE Micro* 42, 1 (2022), 116–127. https://doi.org/10.1109/MM.2021.3097700

[26] Thomas N Kipf and Max Welling. 2016. Semi-supervised classification with graph convolutional networks. *arXiv preprint arXiv:1609.02907* (2016).

[27] Youngeun Kwon, Yunjae Lee, and Minsoo Rhu. 2019. TensorDIMM: A Practical Near-Memory Processing Architecture for Embeddings and Tensor Operations in Deep Learning. In *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture* (Columbus, OH, USA) *(MICRO '52)*. Association for Computing Machinery, New York, NY, USA, 740–753. https://doi.org/10.1145/3352460.3358284

[28] Adam Lerer, Ledell Wu, Jiajun Shen, Timothee Lacroix, Luca Wehrstedt, Abhijit Bose, and Alex Peysakhovich. 2019. Pytorch-BigGraph: A Large Scale Graph Embedding System. In *Proceedings of Machine Learning and Systems*, A. Talwalkar, V. Smith, and M. Zaharia (Eds.), Vol. 1. 120–131. https://proceedings.mlsys.org/paper_files/paper/2019/file/1eb34d662b67a14e3511d0dfd78669be-Paper.pdf

[29] Bingbing Li, Santosh Pandey, Haowen Fang, Yanjun Lyv, Ji Li, Jieyang Chen, Mimi Xie, Lipeng Wan, Hang Liu, and Caiwen Ding. 2020. FTRANS: energy-efficient acceleration of transformers using FPGA. In *Proceedings of the ACM/IEEE International Symposium on Low Power Electronics and Design* (Boston, Massachusetts) *(ISLPED '20)*. Association for Computing Machinery, New York, NY, USA, 175–180. https://doi.org/10.1145/3370748.3406567

[30] Jiajun Li, Ahmed Louri, Avinash Karanth, and Razvan Bunescu. 2021. GCNAX: A Flexible and Energy-efficient Accelerator for Graph Convolutional Neural Networks. In *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. 775–788. https://doi.org/10.1109/HPCA51647.2021.00070

[31] Shang Li, Zhiyuan Yang, Dhiraj Reddy, Ankur Srivastava, and Bruce Jacob. 2020. DRAMsim3: A Cycle-Accurate, Thermal-Capable DRAM Simulator. *IEEE Computer Architecture Letters* 19, 2 (2020), 106–109. https://doi.org/10.1109/LCA.2020.2973991

[32] Yuquan Li, Chang-Yu Hsieh, Ruiqiang Lu, Xiaoqing Gong, Xiaorui Wang, Pengyong Li, Shuo Liu, Yanan Tian, Dejun Jiang, Jiaxian Yan, et al. 2022. An adaptive graph learning method for automated molecular interactions and properties predictions. *nature machine intelligence* 4, 7 (2022), 645–651.

[33] Yuhan Li, Peisong Wang, Zhixun Li, Jeffrey Xu Yu, and Jia Li. 2024. ZeroG: Investigating Cross-dataset Zero-shot Transferability in Graphs. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining* (Barcelona, Spain) *(KDD '24)*. Association for Computing Machinery, New York, NY, USA, 1725–1735. https://doi.org/10.1145/3637528.3671982

[34] Shengwen Liang, Ying Wang, Cheng Liu, Lei He, Huawei LI, Dawen Xu, and Xiaowei Li. 2021. EnGN: A High-Throughput and Energy-Efficient Accelerator for Large Graph Neural Networks. *IEEE Trans. Comput.* 70, 9 (2021), 1511–1525. https://doi.org/10.1109/TC.2020.3014632

[35] Yi-Chien Lin, Bingyi Zhang, and Viktor Prasanna. 2022. HP-GNN: Generating High Throughput GNN Training Implementation on CPU-FPGA Heterogeneous Platform. In *Proceedings of the 2022 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays* (Virtual Event, USA) *(FPGA '22)*. Association for Computing Machinery, New York, NY, USA, 123–133. https://doi.org/10.1145/3490422.3502359

[36] Hao Liu, Jiarui Feng, Lecheng Kong, Ningyue Liang, Dacheng Tao, Yixin Chen, and Muhan Zhang. 2023. One for all: Towards training one graph model for all classification tasks. *arXiv preprint arXiv:2310.00149* (2023).

[37] Liqiang Lu, Yicheng Jin, Hangrui Bi, Zizhang Luo, Peng Li, Tao Wang, and Yun Liang. 2021. Sanger: A Co-Design Framework for Enabling Sparse Attention using Reconfigurable Architecture. In *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture* (Virtual Event, Greece) *(MICRO '21)*. Association for Computing Machinery, New York, NY, USA, 977–991. https://doi.org/10.1145/3466752.3480125

[38] Andrew Kachites McCallum, Kamal Nigam, Jason Rennie, and Kristie Seymore. 2000. Automating the construction of internet portals with machine learning. *Information Retrieval* 3 (2000), 127–163.

[39] Jaehyun Park, Byeongho Kim, Sungmin Yun, Eojin Lee, Minsoo Rhu, and Jung Ho Ahn. 2021. TRiM: Enhancing Processor-Memory Interfaces with Scalable Tensor Reduction in Memory. In *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture* (Virtual Event, Greece) *(MICRO '21)*. Association for Computing Machinery, New York, NY, USA, 268–281.

https://doi.org/10.1145/3466752.3480080

[40] Yubin Qin, Yang Wang, Dazheng Deng, Zhiren Zhao, Xiaolong Yang, Leibo Liu, Shaojun Wei, Yang Hu, and Shouyi Yin. 2023. FACT: FFN-Attention Co-optimized Transformer Architecture with Eager Correlation Prediction. In *Proceedings of the 50th Annual International Symposium on Computer Architecture* (Orlando, FL, USA) *(ISCA '23)*. Association for Computing Machinery, New York, NY, USA, Article 22, 14 pages. https://doi.org/10.1145/3579371.3589057

[41] Zheng Qu, Liu Liu, Fengbin Tu, Zhaodong Chen, Yufei Ding, and Yuan Xie. 2022. DOTA: detect and omit weak attentions for scalable transformer acceleration. In *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems* (Lausanne, Switzerland) *(ASPLOS '22)*. Association for Computing Machinery, New York, NY, USA, 14–26. https://doi.org/10.1145/3503222.3507738

[42] Nils Reimers and Iryna Gurevych. 2019. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, Kentaro Inui, Jing Jiang, Vincent Ng, and Xiaojun Wan (Eds.). Association for Computational Linguistics, Hong Kong, China, 3982–3992. https://doi.org/10.18653/v1/D19-1410

[43] Satyabrata Sarangi and Bevan Baas. 2021. DeepScaleTool: A Tool for the Accurate Estimation of Technology Scaling in the Deep-Submicron Era. In *2021 IEEE International Symposium on Circuits and Systems (ISCAS)*. 1–5. https://doi.org/10.1109/ISCAS51556.2021.9401196

[44] Rishov Sarkar, Stefan Abi-Karam, Yuqi He, Lakshmi Sathidevi, and Cong Hao. 2023. FlowGNN: A Dataflow Architecture for Real-Time Workload-Agnostic Graph Neural Network Inference. In *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. 1099–1112. https://doi.org/10.1109/HPCA56546.2023.10071015

[45] Franco Scarselli, Marco Gori, Ah Chung Tsoi, Markus Hagenbuchner, and Gabriele Monfardini. 2009. The Graph Neural Network Model. *IEEE Transactions on Neural Networks* 20, 1 (2009), 61–80. https://doi.org/10.1109/TNN.2008.2005605

[46] Michael Schlichtkrull, Thomas N. Kipf, Peter Bloem, Rianne van den Berg, Ivan Titov, and Max Welling. 2018. Modeling Relational Data with Graph Convolutional Networks. In *The Semantic Web*, Aldo Gangemi, Roberto Navigli, Maria-Esther Vidal, Pascal Hitzler, Raphaël Troncy, Laura Hollink, Anna Tordai, and Mehwish Alam (Eds.). Springer International Publishing, Cham, 593–607.

[47] Prithviraj Sen, Galileo Namata, Mustafa Bilgic, Lise Getoor, Brian Galligher, and Tina Eliassi-Rad. 2008. Collective Classification in Network Data. *AI Magazine* 29, 3 (Sep. 2008), 93. https://doi.org/10.1609/aimag.v29i3.2157

[48] Daniel S. Soper. 2023. Hyperparameter Optimization Using Successive Halving with Greedy Cross Validation. *Algorithms* 16, 1 (2023). https://doi.org/10.3390/a16010017

[49] Thierry Tambe, Coleman Hooper, Lillian Pentecost, Tianyu Jia, En-Yu Yang, Marco Donato, Victor Sanh, Paul Whatmough, Alexander M. Rush, David Brooks, and Gu-Yeon Wei. 2021. EdgeBERT: Sentence-Level Energy Optimizations for Latency-Aware Multi-Task NLP Inference. In *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture* (Virtual Event, Greece) *(MICRO '21)*. Association for Computing Machinery, New York, NY, USA, 830–844. https://doi.org/10.1145/3466752.3480095

[50] Teng Tian, Xiaotian Wang, Letian Zhao, Wei Wu, Xuecang Zhang, Fangmin Lu, Tianqi Wang, and Xi Jin. 2022. G-NMP: Accelerating Graph Neural Networks with DIMM-based Near-Memory Processing. *Journal of Systems Architecture* 129 (2022), 102602. https://doi.org/10.1016/j.sysarc.2022.102602

[51] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Ł ukasz Kaiser, and Illia Polosukhin. 2017. Attention is All you Need. In *Advances in Neural Information Processing Systems*, I. Guyon, U. Von Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett (Eds.), Vol. 30. Curran Associates, Inc. https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf

[52] Petar Velickovic, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Lio, Yoshua Bengio, et al. 2017. Graph attention networks. *stat* 1050, 20 (2017), 10–48550.

[53] Hanrui Wang, Zhekai Zhang, and Song Han. 2021. SpAtten: Efficient Sparse Attention Architecture with Cascade Token and Head Pruning. In *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. 97–110. https://doi.org/10.1109/HPCA51647.2021.00018

[54] Yang Wang, Yubin Qin, Dazheng Deng, Jingchuan Wei, Yang Zhou, Yuanqi Fan, Tianbao Chen, Hao Sun, Leibo Liu, Shaojun Wei, and Shouyi Yin. 2023. An Energy-Efficient Transformer Processor Exploiting Dynamic Weak Relevances in Global Attention. *IEEE Journal of Solid-State Circuits* 58, 1 (2023), 227–242. https://doi.org/10.1109/JSSC.2022.3213521

[55] Mingyu Yan, Lei Deng, Xing Hu, Ling Liang, Yujing Feng, Xiaochun Ye, Zhimin Zhang, Dongrui Fan, and Yuan Xie. 2020. HyGCN: A GCN Accelerator with Hybrid Architecture. In *2020 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. 15–29. https://doi.org/10.1109/HPCA47549.2020.00012

[56] Zuoxi Yang and Shoubin Dong. 2020. HAGERec: Hierarchical Attention Graph Convolutional Network Incorporating Knowledge Graph for Explainable Recommendation. *Knowledge-Based Systems* 204 (2020), 106194. https://doi.org/10.1016/j.knosys.2020.106194

[57] Mingi Yoo, Jaeyong Song, Jounghoo Lee, Namhyung Kim, Youngsok Kim, and Jinho Lee. 2023. SGCN: Exploiting Compressed-Sparse Features in Deep Graph Convolutional Network Accelerators. In *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. 1–14. https://doi.org/10.1109/HPCA56546.2023.10071102

[58] Haoran You, Tong Geng, Yongan Zhang, Ang Li, and Yingyan Lin. 2022. GCoD: Graph Convolutional Network Acceleration via Dedicated Algorithm and Accelerator Co-Design. In *2022 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. 460–474. https://doi.org/10.1109/HPCA53966.2022.00041

[59] Sungmin Yun, Hwayong Nam, Jaehyun Park, Byeongho Kim, Jung Ho Ahn, and Eojin Lee. 2024. GraNDe: Efficient Near-Data Processing Architecture for Graph Neural Networks. *IEEE Trans. Comput.* 73, 10 (2024), 2391–2404. https://doi.org/10.1109/TC.2023.3283677

[60] Hanqing Zeng and Viktor Prasanna. 2020. GraphACT: Accelerating GCN Training on CPU-FPGA Heterogeneous Platforms. In *Proceedings of the 2020 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays* (Seaside, CA, USA) *(FPGA '20)*. Association for Computing Machinery, New York, NY, USA, 255–265. https://doi.org/10.1145/3373087.3375312

[61] Shulin Zeng, Jun Liu, Guohao Dai, Xinhao Yang, Tianyu Fu, Hongyi Wang, Wenheng Ma, Hanbo Sun, Shiyao Li, Zixiao Huang, Yadong Dai, Jintao Li, Zehao Wang, Ruoyu Zhang, Kairui Wen, Xuefei Ning, and Yu Wang. 2024. FlightLLM: Efficient Large Language Model Inference with a Complete Mapping Flow on FPGAs. In *Proceedings of the 2024 ACM/SIGDA International Symposium on Field Programmable Gate Arrays* (Monterey, CA, USA) *(FPGA '24)*. Association for Computing Machinery, New York, NY, USA, 223–234. https://doi.org/10.1145/3626202.3637562

[62] Xuan Zhang and Kevin Duh. 2024. Best Practices of Successive Halving on Neural Machine Translation and Large Language Models. In *Proceedings of the 16th biennial conference of the Association for Machine Translation in the Americas (Volume 1: Research Track)*.

[63] Zhe Zhou, Cong Li, Xuechao Wei, Xiaoyang Wang, and Guangyu Sun. 2023. GNNear: Accelerating Full-Batch Training of Graph Neural Networks with near-Memory Processing. In *Proceedings of the International Conference on Parallel Architectures and Compilation Techniques* (Chicago, Illinois) *(PACT '22)*. Association for Computing Machinery, New York, NY, USA, 54–68. https://doi.org/10.1145/3559009.3569670

[64] Yun Zhu, Yaoke Wang, Haizhou Shi, and Siliang Tang. 2024. Efficient tuning and inference for large language models on textual graphs. *arXiv preprint arXiv:2401.15569* (2024).

[65] Zeyu Zhu, Fanrong Li, Gang Li, Zejian Liu, Zitao Mo, Qinghao Hu, Xiaoyao Liang, and Jian Cheng. 2024. MEGA: A Memory-Efficient GNN Accelerator Exploiting Degree-Aware Mixed-Precision Quantization. In *2024 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. 124–138. https://doi.org/10.1109/HPCA57654.2024.00020